Git Bash to GitHub Workflow

Command: pwd
Use: Current folder/location check করার জন্য
Description: Print Working Directory

Step-by-Step Folder Setup in Git Bash (C/D/E Drive)
Step 1: Move to D drive
Command: cd /d (cd → Change Directory)

Step 2: Create a folder
Command: mkdir bongoDev (mkdir → Make folder)

Step 3: Enter the folder
Command: cd bongoDev

Step 4: Check current directory
Command: pwd

Go one step back to the previous folder (parent directory)
Command: cd .. ("dot dot = back")
[NOTE:

1. ধরো তুমি এখন এখানে আছো:
   → /d/bongoDev/project1

2. এখন যদি লিখো:
   → cd ..

3. তাহলে যাবে:
   → /d/bongoDev

4. আর আবার cd .. দিলে:
   → /d

cd .. → এক ধাপ পিছনে
cd ../.. → দুই ধাপ পিছনে
]

The (ls) **command is used to display a list of all files and folders inside the current directory.**

Command: ls
Description: list directory contents
[NOTE:
যদি বর্তমান ফোল্ডারে থাকে:
bongoDev
README.md
index.html
test.py

ls কমান্ড চালালে আউটপুট হবে:
bongoDev README.md index.html test.py

Some useful variations:
ls → সাধারণ তালিকা
ls -a → hidden ফাইলসহ সব দেখায়
{
output: . .. bongoDev README.md .gitignore
Notes:
. = current directory
. . = parent directory
}
ls -l → details (size, permission) সহ দেখায়
ls -la →সব ফাইল (hidden + normal) দেখানো সাথে **detailed info**
]

The clear command is also very common in Git Bash. Let me explain it step by step:

Command: clear
[NOTE:
Purpose:
Terminal / Git Bash স্ক্রিন খালি করা
যাতে আগের আউটপুট বা কমান্ডগুলো আর দেখা না যায়
কিন্তু ফাইল বা folder-এর কোনো data মুছে যায় না!
]

Command to go to home directory:

Command: cd ~ [~ = tilde]
Description: Goes to the home directory of the current user.

Creating a new file named README.md in Git Bash:

Command: touch README.md

[NOTE:
touch = নতুন ফাইল তৈরি করা
যদি ফাইল আগে থেকেই থাকে, তাহলে শুধু **last modified time update** করে
.md= Markdown file (যেটা GitHub এ documentation এর জন্য ইউজ হয়)
]

Editing files(`nano / vim`):
Command: nano README.md

[NOTE:
nano editor খুলে তুমি লিখতে পারবে
লিখে **Ctrl+O** → Save, **Ctrl+X** → Exit
]

We use the **cat** command to view what is written inside a file in the Terminal.
Command: cat README.md
[NOTE:
cat README.md (cat filename)
`cat` = **Concatenate**
সাধারণভাবে এটি ফাইলের সম্পূর্ণ কনটেন্ট একসাথে স্ক্রিনে প্রিন্ট করে
ফাইল এডিট করে না, শুধু দেখায়
]

git init কমান্ড ব্যবহার করা হয় একটা সাধারণ ফোল্ডারকে **Git repository** বানানোর জন্য:
Command: git init

[NOTE:

1. এই কমান্ড দিলে Git বুঝে যায় যে - **“এই ফোল্ডারটা এখন থেকে version control এর আওতায়”।**
2.ফোল্ডারের ভিতরে একটি **hidden** `.git` ফোল্ডার তৈরি হয়
3.এই `.git` ফোল্ডারের ভিতরেই:
    - file changes track করা হয়
    - commit history থাকে
    - branch information থাকে
4. `.git` folder থাকলেই folder-টা **Git repository**

]

`git status` কমান্ড ব্যবহার করে আমরা **Git repository-**এর বর্তমান অবস্থা **(current state)** দেখি:

Command: git status

[NOTE:
Untracked files → নতুন ফাইল, যেগুলো Git এখনো track করছে না
Modified files → আগের ফাইলে পরিবর্তন হয়েছে
Staged files → `git add` করা হয়েছে, commit-এর জন্য ready
]

`git add .` কমান্ড ব্যবহার করে আমরা বর্তমান ফোল্ডারের সব নতুন ও পরিবর্তিত ফাইলকে **staging area-**তে পাঠাই:

Command: **git add .**

[NOTE:
1. সব **untracked files** track হতে শুরু করে
2.সব **modified files** staging area-তে যায়
3.ফাইলগুলো commit-এর জন্য **ready** হয়
]

`git reset .` কমান্ড ব্যবহার করা হয় **staging area** থেকে সব ফাইল সরিয়ে দেওয়ার জন্য:
Command: **git reset .**

[NOTE:
ফাইল ডিলিট হয় না

ফাইলের পরিবর্তনও নষ্ট হয় না
শুধু staging area থেকে বের হয়ে যায়
staged files → আবার **modified / untracked** অবস্থায় ফিরে যায়
working directory যেমন ছিল, তেমনই থাকে
]

`git commit -m "message"` ব্যবহার করা হয় **staging area-**তে থাকা ফাইলগুলোর একটি **snapshot (version)** সংরক্ষণ করার জন্য:
Command: `git commit -m "your message"`

[NOTE:
`-m` = **message**
commit message-এ আমরা লিখি:
কী পরিবর্তন করেছি
কেন পরিবর্তন করেছি (সংক্ষেপে)
]

`git log` কমান্ড ব্যবহার করে আমরা **Git repository-**এর সব **commit history** দেখি:
Command: git log

[NOTE:
Commit hash (unique ID)
Author name & email
Date & time
Commit message
]

git config --global user.email (your_github_gmail)@gmail.com - এই কমান্ড ব্যবহার করা হয় **Git-**এ নিজের পরিচয় **(identity)** সেট করার জন্য:

Command: git config --global user.email tachikul@gmail.com

git config --global user.name "your_github_name" - এই কমান্ড ব্যবহার করা হয় **Git-**এ নিজের নাম সেট করার জন্য:
git config --global user.name Tachikul Islam
`git log -p` কমান্ড ব্যবহার করে আমরা **commit history** এর সাথে প্রতিটি **commit-**এ কি কি পরিবর্তন হয়েছে **(diff)** তা দেখতে পারি:

Command: git log -p

`git diff` কমান্ড ব্যবহার করে আমরা **working directory** এবং **staging area-**তে থাকা পরিবর্তনগুলোর পার্থক্য **(difference)** দেখতে পারি।

Command: git diff

[NOTE:
কোন ফাইলের কোন লাইন **added (+)** বা **removed (-)** হয়েছে
এটা **staging area-**তে **commit** করার আগে চেক করার জন্য perfect
]

Github a SSH key setup:
Command: git remote add Tachikul (SSH key link copy and then paste)

`git remote -v` কমান্ড ব্যবহার করে আমরা **local Git repository-**এর সাথে সংযুক্ত সব **remote repository-**এর **URL** দেখতে পারি:

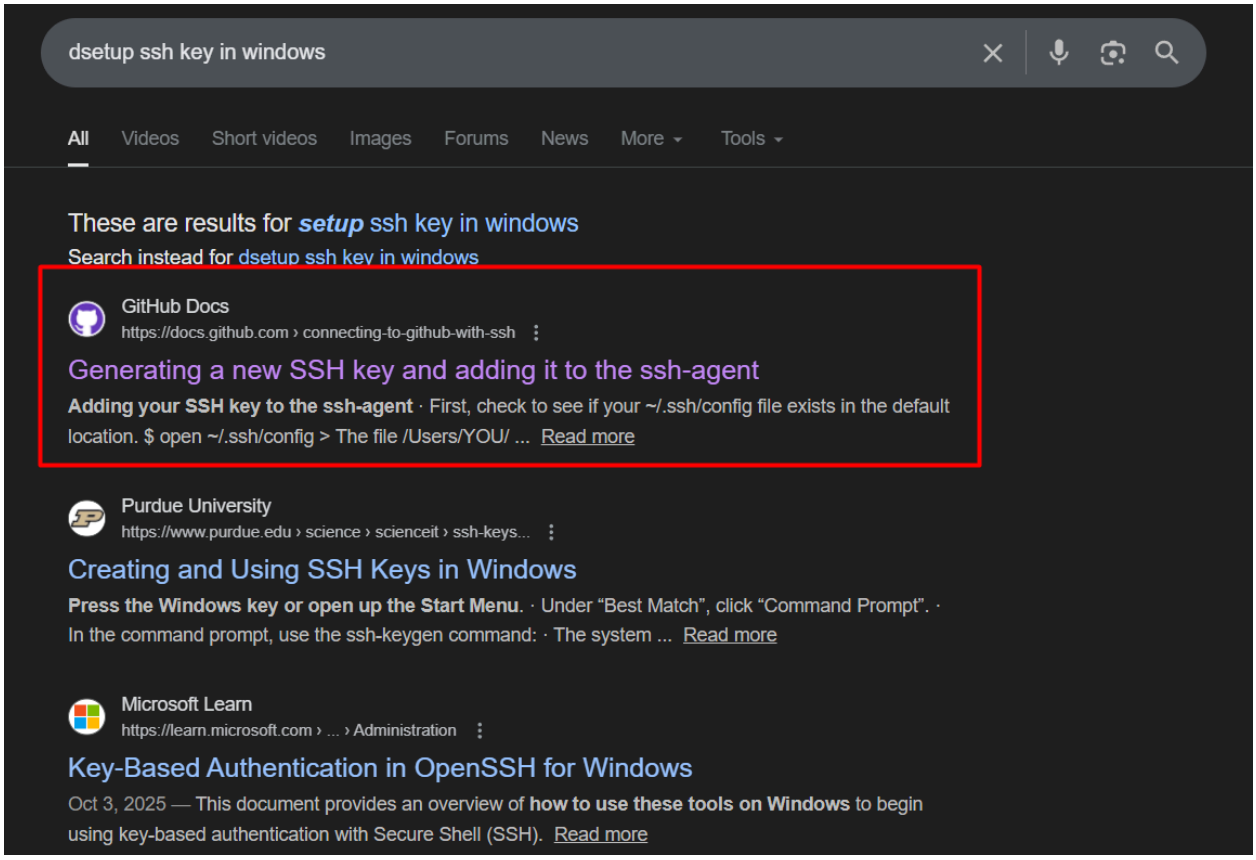
Command: git remote -v

`git push` কমান্ড ব্যবহার করে আমরা **local Git repository-**এর **commit**গুলো **remote repository (GitHub, GitLab** ইত্যাদি**)-**এ পাঠাই:

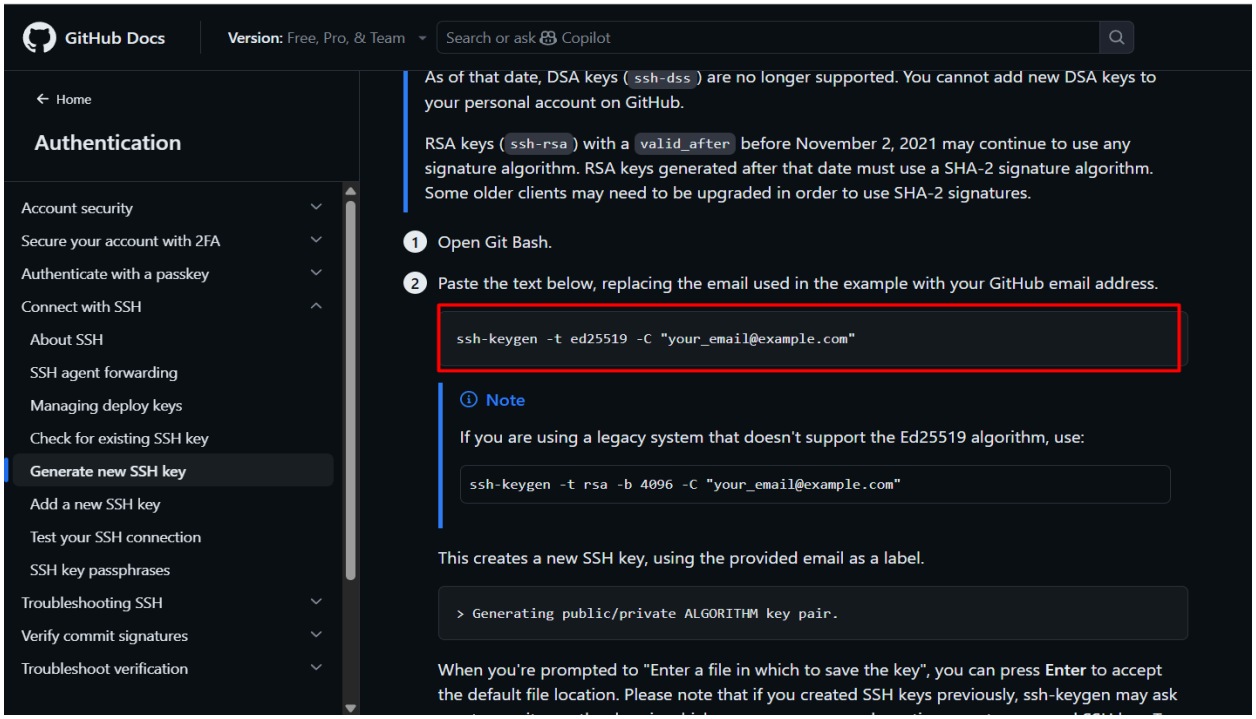
Command: git push Tachikul master (git push <remote-name> <branch-name>)

[NOTE:
git push - Local commits remote-এ push করা
Tachikul - Remote repository-এর nickname (যেমন `origin`)
Master - Remote branch-এ push করা (অধিকাংশ ক্ষেত্রে main branch)
]

Search google - Setup SSH key in windows then



Then copy the link/command



Then go to file explorer